

TOPIC 42/60

What is Chunking in RAG?

The Foundation of Vector Search.

How breaking massive documents into bite-sized pieces dramatically multiplies AI accuracy.

Ashish Jangra

linkedin.com/in/ashish-jangra



THE PROBLEM

The 10,000 Page PDF

You cannot feed an entire corporate database into a single AI prompt. Even if you could, the accuracy would collapse due to information overload.

The "Lost in the Middle" Effect

When you give an LLM a massive block of text, it tends to remember the very beginning and the very end, while completely ignoring critical facts buried in the middle. Furthermore, **embedding models** have strict token limits (often maxing out at 512 or 8192 tokens).

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



Slicing the Data



The Goal:

Ashish Jangra
[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



STRATEGY 1

Fixed-Size Chunking

The most common and simplistic method. You slice the text exactly every N characters or tokens.



The Pros

Incredibly fast to compute. Very easy to implement. Creates perfectly uniform vector sizes.



The Cons (Context Shearing)

It is blind to meaning. It might cut a crucial sentence directly in half, completely destroying the semantic context of both resulting chunks.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



THE ENGINEERING FIX

Chunk Overlap

To prevent "context shearing" in fixed-size chunking, engineers introduce an **overlap window** (usually 10% to 20%).

Original Text: "The AI agent failed because the server crashed."

Chunk A: "The AI agent failed because the server"

Chunk B: "the server crashed."

** The shared words act as a semantic bridge, ensuring neither chunk loses the core meaning of the event.*

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



STRATEGY 2

Structural Chunking

Instead of relying on arbitrary character counts, we use the natural formatting of the document (Markdown, HTML, or NLP libraries) to guide the splits.

Common Structural Anchors:

Headers: Splitting when an H1, H2, or H3 appears.

¶ Paragraphs: Using regex or NLTK to split at natural line breaks.

{ Code Ensuring a Python script isn't cut in
} Blocks: half.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



STRATEGY 3

Semantic Chunking

The "Holy Grail" of RAG pipeline ingestion. Rather than looking at formatting, it uses a small AI model to evaluate the actual *meaning* of the text.

The algorithm continually calculates the vector similarity between adjacent sentences. When the similarity suddenly drops, it signals that the topic has changed.

Result: Chunks that perfectly encapsulate single, unified ideas. High compute cost upfront, but maximum retrieval accuracy.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



TRADE-OFFS

The Size Dilemma

Chunk Size	Advantage	Drawback
Small (100-250t)	High Precision	Lacks broad context
Medium (500t)	Great Balance	Minor noise risk
Large (2000t+)	Deep Context	Retrieval Dilution

Pro-Tip: Use *Parent-Child Chunking*. Search across Small chunks for extreme accuracy, but pass the Medium "Parent" chunk to the LLM to provide full context!

Ashish Jangra

linkedin.com/in/ashish-jangra



ARCHITECTURE

The Ingestion Flow

1

Raw Documents (PDF, HTML)

2

Chunking Strategy Applied (e.g., Markdown)

3

Embedding Model creates Vectors

4

Stored in Vector DB for fast retrieval

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



Chunking in RAG

CONCEPT UNLOCKED



Prevents context window overload



Fixed-Size requires overlap to bridge gaps



Structural utilizes headers/paragraphs



Semantic splits text precisely when topics change

CONNECT WITH ME



LinkedIn



GitHub



Kaggle



Instagram

Ashish Jangra

linkedin.com/in/ashish-jangra

